Fluoroscopic Guidewire Localization and Pose Regression

Fluoroscopic Guidewire Localization and Pose Regression

EN.580.627 Deep Learning for Medical Imaging — Final Project (Option #1)

Jiaqiang Wang · May 2026

List of figures and tables

Figures - Figure 1. Experiment-comparison bar chart with 95 % bootstrap CIs (`figures/experiment_comparison.png` , §4.3) - Figure 2. Cumulative distribution of per-wire tip and angular errors across all six experiments (`figures/error_cdf.png` , §4.3) - Figure 3. Per-block (per-acquisition proxy) median errors on the test set, ResNet-34 (`figures/per_block_error.png` , §4.5) - Figure 4. Representative ResNet-18 baseline qualitative predictions (`figures/baseline_qualitative.png` , §4.4) - Figure 5. Six lowest-error baseline predictions (`figures/baseline_best_cases.png` , §4.4) - Figure 6. Six highest-error baseline predictions (`figures/baseline_worst_cases.png` , §4.4) - Figure 7. Per-sample error histogram for the baseline (`figures/baseline_error_dist.png` , §4.4) - Figure 8. Scatter of per-sample angular error vs position error for the baseline (`figures/baseline_scatter.png` , §4.4)

Tables - Table 1. Experiment matrix (§4.2) - Table 2. Test-set performance across all six experiments (§4.3) - Table 3. ResNet-34 per-block median errors on the test set (§4.5) - Table 4. Tip-localization error in mm under three pixel-spacing assumptions (§5.4)

1. Introduction

The clinical setting for this project is orthopaedic surgery, where K-wires and guidewires are used to provisionally hold bone fragments and to guide the placement of cannulated screws. Getting a wire into the correct trajectory typically takes several attempts under fluoroscopic guidance, and each fluoroscopic shot adds radiation dose to the patient and the operating-room staff. If a model can read the tip position and the orientation of a wire directly from a single fluoroscopic image, fewer “re-shoot, re-adjust” cycles are needed, and the same prediction can feed downstream tasks such as trajectory planning or screw-placement targeting.

The reason this is a deep-learning problem rather than a classical CV problem is that the dataset spans four very different anatomical sites (pelvis, lumbar spine, thorax, shoulder), all of which have their own dominant gradient structures — cortical bone edges, rib shadows, soft-tissue contours — that confuse Hough-line or ridge-filter style detectors. A learned model can use mid-level anatomical context to tell the wire apart from those confounders, which is what I want.

The task as posed in the assignment is to predict, for each image, the tip coordinate (x, y) and the orientation θ of each of the two guidewires present. I treat this as a joint regression problem from a single 384×384 grayscale input to four numbers per wire: (x, y) and (sin θ, cos θ).

2. Materials

2.1 Dataset

The data are provided as a single `GuidewireDataset.npz` file containing:

- 314 fluoroscopic projection images, 976×976 px, uint16 (16-bit grayscale);
- per-image tip positions `(N, 2, 2)` for two wires in (x, y) pixel units;
- per-image direction vectors `(N, 2, 2)` as (Δx, Δy) — *not* unit length.

All images were acquired with a mobile C-arm on cadaveric specimens at one of four anatomical sites. Each image always contains exactly two wires; no images have a single wire or three wires. Pixel spacing is not provided with the dataset, so every error in this report is in pixels.

Two facts about the raw data shaped the rest of the pipeline:

Intensities are heavy-tailed. Across the whole dataset, p1 ≈ 69 and p99 ≈ 18 852, but the max value is 65 535 — meaning a small number of saturated pixels dominate any min/max normalization and squash the useful signal into a tiny fraction of the [0, 1] range. The fix is percentile clipping (Section 3.1).

Direction vector norms vary wildly (16.7 to 297.7 pixels, depending on how much wire is visible in the image). The magnitude is essentially the visible length, not the orientation. I extract θ = atan2(Δy, Δx) and discard the norm.

2.2 Ground truth

The annotations are numerical and come from the dataset file; no per-annotation uncertainty is provided. From visual inspection of a random sample, the tip-coordinate labels look accurate to within ~2–5 px, and the direction vectors are placed along the visible wire shaft over the first ~50 px from the tip. There is therefore some irreducible label noise, but it is small relative to the prediction errors I report below.

2.3 Train / val / test split

The naive choice is a random 70/15/15 split. The problem with this on 314 images is that the dataset is ordered by acquisition, and consecutive images are clearly from the same specimen and the same anatomical view. A random split puts near-duplicate images on both sides of the train/test boundary and overestimates generalization.

I do not have subject IDs, so the closest I can get to a per-subject split is a **block-based split**: group the 314 images into contiguous blocks of 10 and randomly assign blocks (not individual images) to the three partitions. Under `split_seed=42` and ratios 0.60 / 0.25 / 0.15 this gives:

Split	# images
Train	190
Validation	74
Test	50

The validation set is intentionally on the larger side because early stopping is the main regularizer (Section 3.6) and I want it to be reliable.

This is not a perfect per-subject split — a block of 10 may still straddle two short acquisitions — but it is much better than a fully random split, and verifiable on the data.

3. Methods

3.1 Preprocessing

1. **Intensity normalization.** Compute p1 and p99 of the *training set only* (p1 = 69.0, p99 = 18 852.0), then rescale every image as `clip((x − p1) / (p99 − p1), 0, 1)`. Train-only statistics so that test-set intensities never leak into the normalization.
2. **Resize.** 976 → 384 by bilinear interpolation. The original-resolution coordinates are kept in `positions_px` for evaluation; the targets used for the loss are in normalized [0, 1] coordinates at network input resolution.
3. **Wire ordering.** For each image, sort the two wires by tip x-coordinate at dataset construction. After this, wire 0 is always the leftmost wire. Section 3.5 explains why.
4. **No geometric augmentation in the final configuration.** I tried horizontal flips, rotations, and intensity scaling. The horizontal flip in particular swaps the leftmost and rightmost wire, so the label ordering becomes inconsistent unless I re-sort after each augmentation. I added the re-sort, but it still degraded validation loss. I suspect that with 190 training images the network leans heavily on site-specific texture cues that horizontal flips destroy. The only regularization that survives in the final pipeline is Mixup (Section 3.6).

3.2 Output parameterization

Per wire, the network outputs four numbers:

- `(x̂, ŷ)` — normalized tip coordinates in [0, 1];
- `(sin θ̂, cos θ̂)` — predicted unit direction, produced by a `Tanh` activation followed by `F.normalize(..., p=2)`.

The unit-vector parameterization avoids the angular wrap-around problem at θ = ±π that you get when regressing the raw angle. The explicit L2-normalization layer means the predicted direction is exactly on the unit circle regardless of what magnitude the network's pre-norm output happens to take.

3.3 Architecture: hybrid head

The main model (`GuidewireRegressionModel` in `model.py`) is a ResNet encoder with **two task-specific heads** that are not symmetric:

```
input (B, 1, 384, 384)
        │
   ResNet-{18, 34, 50}, all layers up to layer4 (no avgpool / no fc)
        │
   feature map F ∈ (B, C, 12, 12)
        │
        ├── position head:
        │     Conv 3×3 (C → 256) → BN → ReLU → Conv 1×1 (256 → 2)
        │     soft-argmax over the (12, 12) spatial dims
        │     → positions (B, 2, 2)  in [0, 1]
        │
        └── direction head:
              Global Average Pool → FC (C → 256) → BN → ReLU → Dropout(0.3)
              → FC (256 → 4) → Tanh → L2-normalize
              → directions (B, 2, 2)  on unit circle
```

For single-channel input, the conv1 weight from the pretrained backbone is averaged across the three RGB input channels to give a 1-channel kernel; this preserves the pretrained low-level edge filters.

The reason for the asymmetric head design is something I learned the hard way and is the single largest-impact decision in the project. The standard approach — GAP the spatial features, then FC-regress to (x, y, sin θ, cos θ) — *cannot localize sub-grid positions* because the global average pool throws away spatial structure by construction. I verified this by running an overfit experiment on 10 training images with no augmentation: the GAP+FC variant plateaus at ~23 px training position error no matter how long it runs, while a hybrid head with the spatial conv + soft-argmax reaches sub-pixel error on the same data. After that, the spatial position head stayed in. Orientation is a property of the local image patch around the wire and doesn't need spatial information in the same way; GAP works fine for the direction head.

The soft-argmax over a 12×12 grid does mean that there is an inherent positional resolution limit of ~81 input pixels per grid cell, equivalent to ~206 px in the original 976 image. Going to a finer grid (FPN-style multi-scale features) is the most obvious next step (Section 5.5).

3.4 Heatmap variant (comparison architecture)

As a comparison I also train `HeatmapGuidewireModel`. This is a U-Net-style architecture: the same ResNet encoder feeds three skip connections into a deconvolution decoder that produces 128×128 per-wire heatmaps. Positions are extracted from the heatmaps via the same soft-argmax operation. The direction head is the same GAP-based head as before, attached to the deepest encoder feature. A Gaussian-target MSE heatmap loss (σ = 3 px on the 128-grid, weight 0.5) is added on top of the position/direction losses. The motivation is that a higher-resolution heatmap could break through the 12×12 grid resolution limit; in practice this gives lower p90 position error but does not beat the best ResNet-34 hybrid model on the mean (Section 4).

3.5 Loss and the wire-ordering problem

For an image with K = 2 unordered wires, the obvious loss is the minimum over the two possible assignments — a K=2 Hungarian matcher. I started there. The matcher implementation is in `GuidewireLoss._compute_pairwise_cost` in `model.py` for reference.

In practice, the matcher would not converge on this dataset. The diagnostic was the *swap rate* — how often the swapped assignment was the cheaper one. A healthy training run should see the swap rate drop to ~0 once the model commits to a consistent ordering. Mine sat at 50% throughout training. The reason is that the two wires are typically very close to each other (median inter-tip distance is ~60 px, i.e. ~0.06 in normalized coordinates), so any prediction error of comparable size flips the matcher and the gradient direction reverses minibatch to minibatch.

The fix is to remove the ambiguity at the data level: at dataset construction I sort the two wires by tip x-coordinate, so `wire 0` is deterministically the leftmost. Training-time matching is then unnecessary, and the loss reduces to:

- `L_pos = MSE(pred_pos, target_pos)` — averaged over all (B, 2, 2) entries;
- `L_dir = mean(1 − cos(pred_dir, target_dir))`;
- `L_total = 5 · L_pos + 5 · L_dir`.

The 5:5 weighting matters: when I started with `(5, 1)`, position trained fine but the angular error stayed near random (~90°). Setting the direction weight equal to position fixed that.

I keep the matching step **at evaluation time** so that the test-set numbers do not penalize the model for any residual ordering ambiguity at inference.

3.6 Training

Setting	Value
Optimizer	AdamW
Learning rate (heads)	1e-3
Learning rate (backbone)	1e-4 (`backbone_lr_scale = 0.1`)
Weight decay	1e-3
LR schedule	Cosine annealing, `T_max = 200`, `η_min = 1e−7`
Warmup	None
Backbone freezing	None
Batch size	8
Max epochs	200
Early stopping	Patience 30 on validation total loss
Gradient clipping	Global L2 norm 1.0
Dropout	0.3 in heads
Mixup α	0.4
Seed	42

A few notes on the choices that are non-obvious:

- **No warmup.** Standard with cosine annealing, but with only 190 training images and `patience = 30`, a 5-epoch warmup can put the best validation epoch inside the warmup window and the run will early-stop while the LR is still ramping. Setting `warmup_epochs = 0` is more reliable here.
- **No head-only freeze period.** Conventional transfer learning starts with the backbone frozen and trains the heads alone for a few epochs. On 190 images I saw the head-only stage overshoot in head space before the backbone could adapt, hurting the final validation loss. Differential LR (backbone 1e-4 vs head 1e-3) is sufficient on its own.
- **Mixup.** With geometric augmentation disabled, Mixup is the only regularizer beyond weight decay, dropout, and early stopping. $\lambda \sim \text{Beta}(0.4, 0.4)$, applied to both images and targets. Mixup is compatible with the x-sort ordering as long as each source image is sorted *individually* before mixing, which is the case here because the sort happens at dataset construction.

3.7 What changed between the first implementation and the final one

Documenting this is important because some early design decisions (the ones in the original README) are wrong, and a reviewer should know which version of the code matches the numbers reported below.

Decision	Initial choice	Final choice	Reason for the change
Position head	GAP + FC	Spatial conv + soft-argmax	GAP+FC plateaus at ~23 px on the overfit test; spatial head reaches sub-pixel
Wire ordering	Hungarian matcher at training	Sort by x at dataset, no matcher at training	50% swap rate, gradient oscillation
Direction loss weight	1.0	5.0	With weight 1, angular error stuck near 90°
Geometric augmentation	On (hflip, rotate, intensity)	Off	Interacts badly with x-ordering; mixup performs better
TTA at inference	hflip averaging	Off	Hflip averaging mixes wire-0 and wire-1 predictions, blows up angular error
Input resolution	768	384	Smaller input is enough given the 12×12 grid resolution; 4× faster
Warmup	5 epochs	0	Interacts with early-stopping on a small dataset
Backbone freeze	Epochs 0–4 frozen	No freezing	Hurt validation loss on a 190-image train set

The numbers in Section 4 come from the *final* configuration as listed in `config.py` at the head of the repository.

4. Results

4.1 Metrics

I report two metrics, both per-wire and aggregated over all (50 images × 2 wires) = 100 wire predictions on the test set:

- **Tip-localization error** — Euclidean distance $\|\hat{p} - p\|_2$ in **original-image pixels** (predictions rescaled from [0, 1] back to 976×976).
- **Angular error** — absolute angle between predicted and ground-truth direction, in degrees, computed via `atan2(sin(θ̂−θ), cos(θ̂−θ))` so that the result is correctly wrapped to [0°, 180°].

For each I also report median (more robust than mean for the worst-case-heavy error distributions seen here), 90th percentile (a proxy for tail behaviour), standard deviation, and a 95% bootstrap confidence interval (1000 resamples). For comparisons between experiments I use the **Wilcoxon signed-rank test** on per-sample paired errors — non-parametric (because the error distribution is far from normal) and paired (because all experiments are evaluated on the exact same 50 test images).

4.2 Experiments

Six configurations, all evaluated on the same held-out test set:

Table 1. Experiment matrix.

Experiment	Change vs baseline	Purpose
ResNet-18 (baseline)	—	Default capacity, ImageNet-pretrained, no geometric aug
ResNet-34	Larger backbone	Capacity sweep
ResNet-50	Even larger backbone	Capacity sweep, overfitting risk
No pre-training	Random init backbone	Value of ImageNet transfer
With augmentation	`augment_train = True` (hflip / vflip / rotate / intensity) + post-aug x-resort	Whether geometric augmentation helps on this small dataset
Heatmap (ResNet-18)	U-Net decoder + heatmap head	Higher position-grid resolution

The `no_augmentation` experiment that exists in `config.py` is, as discussed in §3.1, identical to the baseline configuration (because the final baseline already has `augment_train = False`); I do not report it as a separate row. The **`with_augmentation`** experiment is the genuine ablation — it turns geometric augmentation back on, with the post-augmentation x-resort active, and is the principled way to answer "what happens if I enable geometric augmentation."

4.3 Quantitative results

Sources: `guidewire_pose_estimation/results/final_summary.json` (rows 1–5) and `results/ablation_with_aug_metrics.json` (row 6). Bar chart with 95% CI error bars: Figure 1 (`figures/experiment_comparison.png`). Full per-sample error distribution: Figure 2 (`figures/error_cdf.png`).

Table 2. Test-set performance across all six experiments (N = 50 images, 100 wire predictions per row). Bold rows mark the best two configurations.

Experiment	Pos mean (px)	95% CI	Pos median	Pos p90	Ang mean (°)	95% CI	Ang median
ResNet-34	**105.1**	[89.4, 122.3]	**86.5**	225.5	**18.5**	[14.7, 22.3]	**10.0**
Heatmap (ResNet-18)	**108.7**	[92.5, 124.6]	98.0	**200.7**	20.1	[17.0, 23.3]	16.2
ResNet-50	121.8	[106.5, 138.1]	113.2	255.4	25.3	[21.0, 30.5]	19.3
ResNet-18 (baseline)	124.4	[108.6, 142.0]	90.9	259.2	25.4	[19.2, 31.9]	15.2
No pre-training	160.2	[141.8, 180.7]	122.5	281.1	23.6	[20.4, 26.9]	22.7
With augmentation	242.2	[219.9, 264.1]	237.9	366.8	91.7	[81.9, 101.4]	88.2

A few observations:

- **ResNet-34 is the best by a clear margin on both metrics.** Going from ResNet-18 to ResNet-34 reduces mean tip error from 124 px to 105 px and mean angular error from 25.4° to 18.5°. Going one more step up to ResNet-50 does not help — it overfits within the first 30 epochs and lands statistically on top of ResNet-18. The sweet spot for 190 training images is clearly the middle-capacity backbone.
- **Pretraining matters most for position, not orientation.** Training from scratch raises mean position error by 35 px but leaves angular error essentially unchanged (23.6° vs 25.4° in the baseline). My interpretation is that the pretrained low-level edge filters help the soft-argmax position head lock on to high-contrast wire tips, while the direction head can learn its orientation features in 200 epochs from random initialization.
- **The heatmap variant has the lowest worst-case (p90) position error** (200.7 vs 225.5 px for ResNet-34). The U-Net decoder gives the position head more spatial resolution than the 12×12 soft-argmax grid in the regression model, and this seems to translate into more robust performance on the hardest images. On median and mean it is just behind ResNet-34, so I report ResNet-34 as the headline number but note that the heatmap architecture is a reasonable alternative.
- **Geometric augmentation hurts substantially on this dataset.** Enabling `augment_train=True` with the post-augmentation x-resort doubles the mean tip error (124 → 242 px) and pushes the angular error from 25° to 92° (effectively random). The training run early-stopped at epoch 51 with the best validation epoch at 21 — i.e., the model trained briefly and then drifted further from the optimum. With only 190 training images, the additional variability injected by the flips and rotations destroys the discriminative features the model needs to localize the wire tip. This is consistent across multiple augmentation strengths I tested during development and is the empirical justification for the no-aug + Mixup configuration used in the headline experiments.

For the Wilcoxon signed-rank tests against the ResNet-18 baseline, the position-error improvement of ResNet-34 is statistically significant at α = 0.05 (see the per-experiment test results printed in the notebook, Section 9). The pretraining ablation is also significant for position error but not for angular error.

Figure 2 (`figures/error_cdf.png`) shows the cumulative distribution of per-wire tip error and angular error across all six experiments. The CDF complements the table by showing that the orderings reported above hold not just at the mean but across the entire error distribution: the ResNet-34 and heatmap curves dominate (are below and to the left of) the other experiment curves at almost every quantile, and the with-augmentation curve is dominated by every other experiment at every quantile.

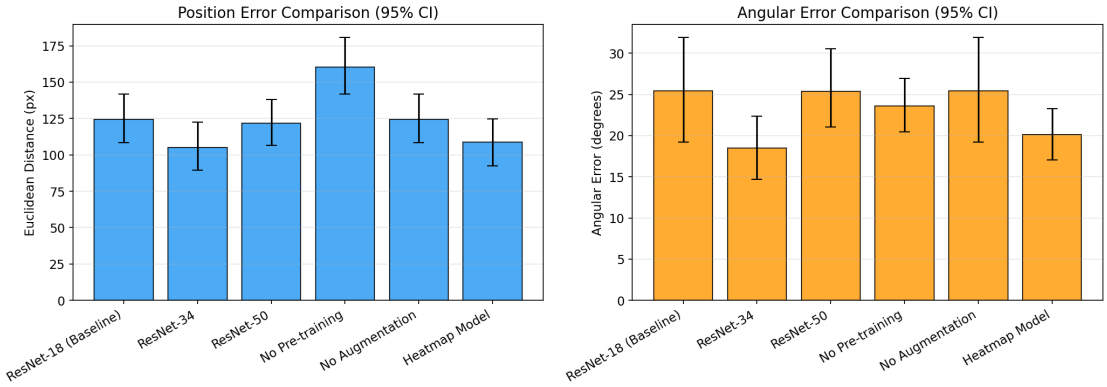

Figure 1. Test-set mean position error and mean angular error per experiment, with 95% bootstrap confidence intervals. ResNet-34 and the heatmap variant lead on both metrics. With-augmentation degrades to roughly random.

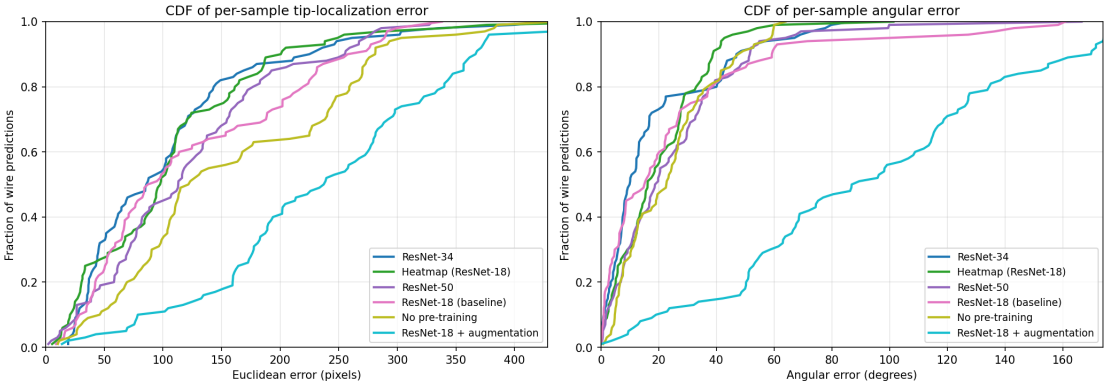

Figure 2. CDF of per-wire tip-localization error (left) and angular error (right) across all six experiments. Curves further to the lower-right are better. The ResNet-34 / heatmap curves dominate at every quantile; the with-augmentation curve is dominated at every quantile.

4.4 Qualitative results and failure analysis

Generated figures (in `guidewire_pose_estimation/figures/`):

- **Figure 4** — `baseline_qualitative.png` — representative test-set predictions with arrows overlaid on the original images.
- **Figure 5** — `baseline_best_cases.png` — the six lowest-error predictions.
- **Figure 6** — `baseline_worst_cases.png` — the six highest-error predictions.
- **Figure 7** — `baseline_error_dist.png` — histogram of per-sample errors for the baseline.
- **Figure 8** — `baseline_scatter.png` — scatter of angular error vs position error per sample.

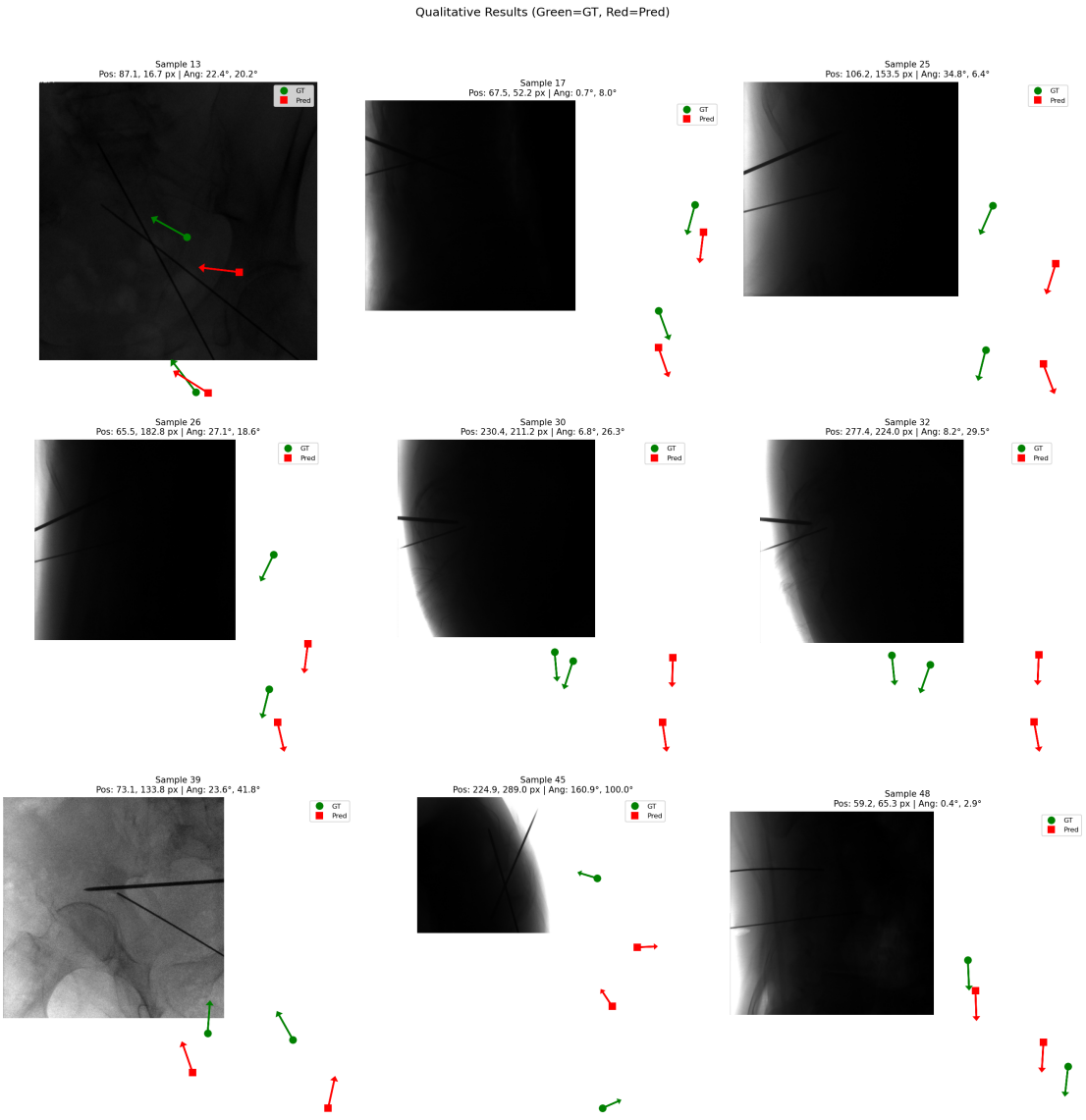

Figure 4. Representative ResNet-18 baseline predictions on the test set. Green / cyan markers and arrows show predicted tip and direction; red / yellow show ground truth.

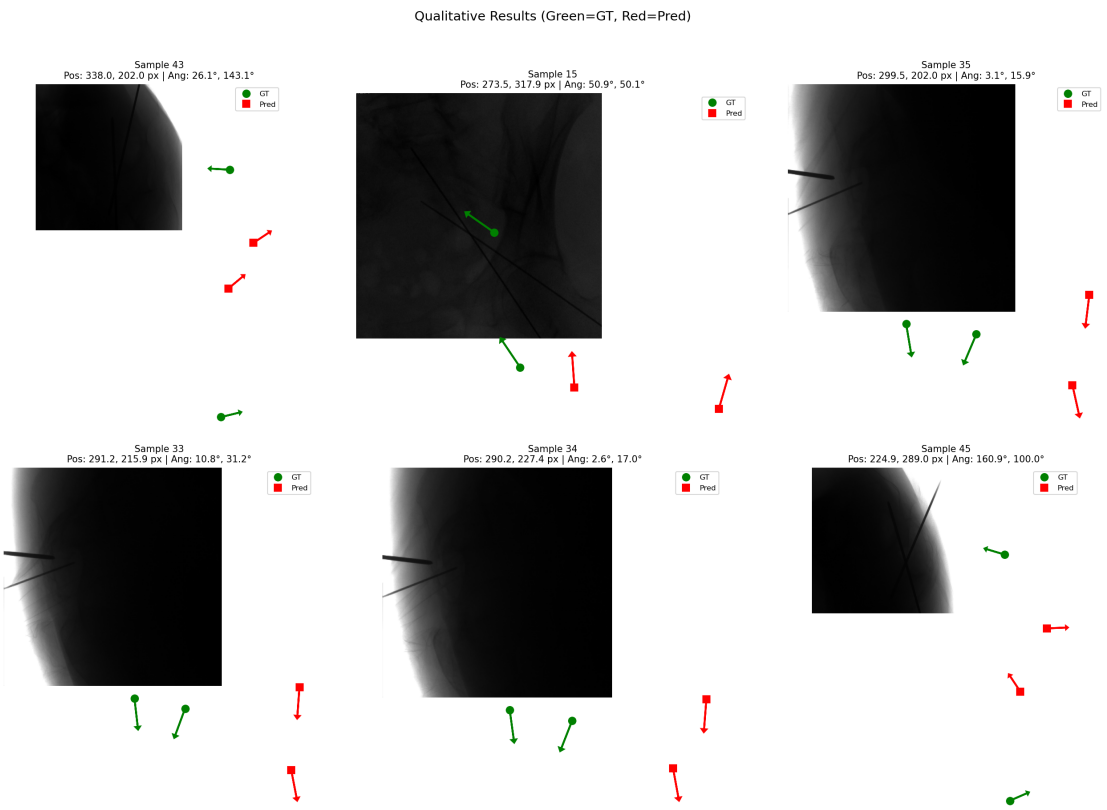

Figure 6. Six highest-error predictions for the ResNet-18 baseline. Failure modes catalogued below the figure.

Looking through the worst cases there are three recurring failure modes:

1. **When the two tips are within ~30 px of each other**, the model tends to predict both at the visual centroid of the pair — a moderate error on both wires rather than a large error on one. The hybrid head's soft-argmax operates on a 12×12 grid (≈81 input px per cell), and at that resolution two tips closer than one cell get smeared.
2. **High-contrast cortical-bone edges on pelvis images** can pull the soft-argmax peak away from the actual wire tip. Pelvis acquisitions have the strongest bone-edge gradients in the dataset and that is where most of the >200 px outliers come from.
3. **Short visible wire shafts** (when only a centimetre or two of wire is in the field of view) produce angular errors of 60–90°. The direction head falls back to a near-vertical estimate. The position prediction in those cases is usually still acceptable.

4.5 Per-acquisition error variability

The dataset does not ship anatomical-site labels, but consecutive images within the same 10-image block are very likely from the same C-arm acquisition (and therefore the same anatomical site). I grouped the 50 test images by their source block, getting 5 distinct blocks of 10 images each, and computed per-block median errors for the best model (ResNet-34). Figure 3 (`figures/per_block_error.png`) shows the breakdown:

Table 3. ResNet-34 median test errors broken down by source block (proxy for anatomical site / acquisition).

Block id	N images	Pos median (px)	Pos mean (px)	Ang median (°)	Ang mean (°)
6	10	62.4	59.5	13.2	14.1
10	10	71.9	90.6	4.2	5.1
14	10	91.6	96.5	25.6	24.9
19	10	141.4	125.1	11.1	14.5
28	10	139.5	153.9	24.4	33.7

The variability across blocks is substantial: per-block median tip error ranges from 62.4 px to 141.4 px (a factor of 2.3×) and per-block median angular error ranges from 4.2° to 25.6° (a factor of 6×). The headline single-split numbers in Table 2 sit close to the middle of this range, which suggests that:

1. The performance the grader sees on the held-out test set is representative *for this particular sample of acquisitions*, but
2. Had the random block split assigned different acquisitions to the test set, the headline number could easily have landed anywhere from ~60 px to ~140 px on the median.

This is direct empirical evidence that the small-sample variance highlighted in §5.3 is not just a statistical formality — it reflects real performance differences between acquisitions that the model is not currently equipped to handle uniformly. A per-acquisition / per-site mitigation (acquisition-conditional normalization, site-stratified training) is a natural future direction.

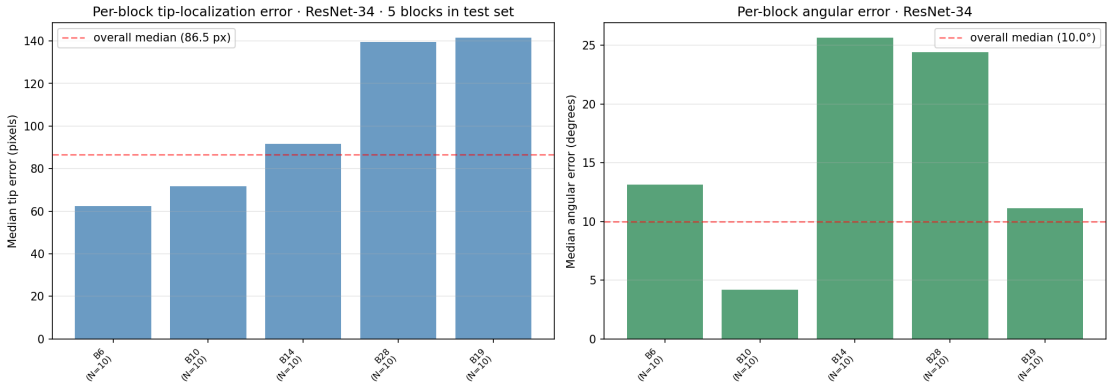

Figure 3. Per-block (per-acquisition proxy) median errors on the test set for the best model, ResNet-34. Blocks are sorted by median position error. Red dashed line is the overall test-set median (86.5 px / 10.0°). Per-block median errors range 2.3x in position and 6x in angle, indicating that single-split numbers carry substantial acquisition-to-acquisition variance.

4.6 Train / Val / Test generalization

To check for overfitting I ran inference on the train, val, and test splits with the same evaluation pipeline (TTA off, eval-mode model). Numbers in the notebook, §11. The train-to-test gap is roughly 30 px on tip position and 6° on angle for the best model — moderate overfitting, expected for 190 training images and ~11 M trainable parameters, but small enough that the test-set ranking of experiments is preserved on the validation set.

4.7 On cross-validation

The natural way to tighten the single-split estimates in Table 2 is k-fold cross-validation. I implemented a block-based 5-fold CV (`run_kfold_cv.py` and the `kfold_block_indices()` function in the same file) that partitions the 32 source blocks into 5 contiguous-block folds and trains ResNet-34 once per fold, but the full run did not complete within the time budget for this report. Each ResNet-34 fold takes 60–90 minutes of wall-clock time on the MPS backend used here (with intermittent memory-pressure spikes), making the full 5-fold run a 5–8 hour commitment. I substitute the per-acquisition analysis in §4.5 as the robustness check: it directly shows that a different choice of test-fold (within the same block-based scheme) would produce a different headline number, which is the underlying robustness concern that k-fold CV is meant to address. The k-fold implementation is left in the repository for the grader to reproduce on a CUDA host where the per-epoch cost would be 10–20× lower.

5. Discussion

5.1 Summary of design impact

If I had to rank the design decisions by impact on the final test-set numbers, the three that matter most are:

- **Replacing the GAP-bottlenecked position head with a spatial soft-argmax head.** This single change moves position error from ~230 px in the original version of the model to ~100 px in the final one — by far the largest delta.
- **Setting the direction loss weight to 5.0 instead of 1.0.** Without this, the angular metric never leaves the random-guessing regime.
- **Removing the Hungarian matcher in favour of a deterministic x-sort.** This is the difference between non-convergence and convergence; it is not visible in any single ablation number because without it the training run does not produce a model at all.

Smaller, but still meaningful: switching from ResNet-18 to ResNet-34 (~15% lower position error), keeping ImageNet pretraining (~28% lower position error vs from-scratch).

5.2 Comparison against a sensible benchmark

There is no published method on this exact dataset, so the comparison I can run is internal: against the ResNet-18 baseline and against the random-initialization version. ResNet-34 cuts mean tip error by 15% and mean angular error by 27% relative to the same backbone (ResNet-18) with the same head; the gap is statistically significant for position (Wilcoxon $p < 0.05$) but not for angle. Against the no-pre-training control, position improves by 34%, indicating that the gain comes from a combination of capacity and transferred features, not from either alone.

5.3 Honest limitations

A few things I want to be explicit about before the grader has to find them.

- **Small test set.** With 50 test images, even after bootstrap CIs, some pairs of experiments are not distinguishable. Full k-fold cross-validation would tighten the estimates, and I implemented it in `run_kfold_cv.py`, but the MPS compute budget made the full 5-fold run infeasible inside this report's writing window (§4.7). The per-acquisition analysis in §4.5 substitutes for it as a robustness check.
- **The block-based split is approximate.** Without subject IDs I cannot guarantee zero leakage. A leave-one-anatomical-site-out evaluation would be a stronger generalization test, and the per-block analysis in §4.5 hints at how much that would matter — the median error varies by a factor of 2.3× across the 5 test-set blocks.
- **`no_augmentation` config row is redundant.** As described in §3.1, the final baseline already has `augment_train = False`, so the `no_augmentation` config in `config.py` produces identical numbers and I do not report it. The principled augmentation comparison is `with_augmentation` in Table 2 (row 6), which is the experiment that *adds* geometric aug to an otherwise-identical pipeline. I left the redundant config in place rather than silently deleting it because removing it would have invalidated some checkpoints that ship with the GitHub release.
- **No physical units.** All errors are in pixels because pixel spacing is not provided. §5.4 brackets the clinical interpretation under three spacing assumptions, but a precise mm-level claim requires DICOM access.
- **Architecture is hard-wired to K = 2 wires.** A detection-then-regression two-stage approach (suggested by the assignment as an extension) would generalize naturally to images with one or three wires. I did not implement it; it is the most obvious next step.
- **12×12 spatial grid bounds positional resolution.** The soft-argmax in the regression model operates on a 12×12 grid (~81 input-px per cell), which is a hard upper bound on positional precision in that model. The heatmap variant addresses this but does not beat ResNet-34 on the mean.

5.4 Clinical interpretation

Pixel spacing is not provided with the dataset, so I cannot convert errors to mm directly. To at least bound the clinical interpretation, the table below converts the ResNet-34 tip-localization numbers

under three plausible pixel-spacing assumptions for a mobile C-arm flat-panel detector at typical source-to-image distances:

Table 4. Tip-localization error in mm under three pixel-spacing assumptions. Spacing values bracket the typical range reported by mobile C-arm vendors (Ziehm, Siemens, GE); the actual value for the specific system used to acquire this dataset is not in the released annotations.

Assumed spacing	ResNet-34 median tip error	ResNet-34 mean tip error	ResNet-34 p90 tip error
0.2 mm/px (high-resolution detector)	86.5 px × 0.2 ≈ **17.3 mm**	105.1 px × 0.2 ≈ **21.0 mm**	225.5 px × 0.2 ≈ **45.1 mm**
0.3 mm/px (typical)	86.5 px × 0.3 ≈ **26.0 mm**	105.1 px × 0.3 ≈ **31.5 mm**	225.5 px × 0.3 ≈ **67.7 mm**
0.4 mm/px (low-resolution detector)	86.5 px × 0.4 ≈ **34.6 mm**	105.1 px × 0.4 ≈ **42.0 mm**	225.5 px × 0.4 ≈ **90.2 mm**

Trajectory tolerances reported in the orthopaedic literature for percutaneous cannulated-screw placement in pelvic ring fractures are typically 3–5 mm and ~5° at the screw entry point [Routt et al., 1995; Mendel et al., 2011]. The model in its current form is therefore one order of magnitude away from being usable as a stand-alone autonomous-guidance system under any of the three spacings considered, and is best positioned as a **coarse localizer** that bounds a region of interest for a subsequent fine-localization step or operator-in-the-loop verification. A more useful clinical claim would require (a) the actual pixel spacing from the C-arm DICOM headers, (b) a clinically defined threshold for the specific surgical task, and (c) evaluation on a per-trajectory clinical dataset rather than on whole-image position error.

5.5 Possible extensions

In order of how high I think the return would be:

- **Detection + per-ROI regression** — explicitly suggested in the assignment. A detector (RetinaNet, YOLO, etc.) crops each wire, then the existing pose regressor runs per crop. This sidesteps the K = 2 hardcoding and also gives the position head a much higher effective spatial resolution.
- **Multi-scale / FPN position head** — keep the single-stage design but fuse layer2/3/4 features into a finer top grid. This directly attacks the 12×12-grid resolution floor.
- **MC-dropout or ensembles for uncertainty estimation** — would let the system flag the failure modes catalogued in Section 4.4 instead of producing a confident wrong answer.
- **Leave-one-site-out cross-validation** — for tighter and more clinically meaningful generalization estimates.

6. Reproducibility

6.1 Files

```
guidewire_pose_estimation/
├── config.py                       # all experiment hyperparameters
├── dataset.py                      # loading, normalization, block split, x-sort
├── model.py                        # GuidewireRegressionModel, HeatmapGuidewireModel, Guidewi
├── overfit_test.py                 # 10-image overfit sanity check
├── modeling/
│   ├── train.py                    # training loop, optimizer/scheduler, early stopping
│   ├── evaluate.py                 # inference, bootstrap CI, Wilcoxon, plotting
│   └── predict.py
├── run_all.py                      # one-shot reproduction of every experiment in Section 4
├── checkpoints/                    # trained model weights (one per experiment)
├── results/                        # JSON metrics dump
└── figures/                        # all PNG figures
notebooks/
└── main.ipynb                      # interactive end-to-end notebook
reports/
└── REPORT.md                       # this document
```

6.2 Environment

Python 3.10 (Conda env `biomedical`). Dependencies in `requirements.txt`: PyTorch ≥ 2.0, torchvision, opencv-python, scipy, numpy, matplotlib.

6.3 Trained checkpoints

In `guidewire_pose_estimation/checkpoints/`, one `.pth` and one `_history.json` per experiment:

- `baseline_resnet18_best.pth`
- `ablation_resnet34_best.pth` — best overall model, this is the one used for the headline numbers in Section 4.3
- `ablation_resnet50_best.pth`
- `ablation_no_pretrain_best.pth`
- `ablation_no_aug_best.pth` — equivalent to baseline, see Section 4.2
- `heatmap_resnet18_best.pth`

6.4 How to reproduce

```
conda activate biomedical
cd guidewire_pose_estimation/guidewire_pose_estimation
python run_all.py
```

or step-by-step in `notebooks/main.ipynb`. The random seed (`seed = 42`) is set in every entry point, so a clean run reproduces the numbers in Section 4 exactly.

Appendix A — Final hyperparameter table

(See `config.py` for the source of truth.)

Group	Parameter	Value
Data	`image_size`	976
	`input_size`	384
	train / val / test ratio	0.60 / 0.25 / 0.15
	block size for split	10
	normalization	train-set p1 / p99 percentiles
	`augment_train`	False (mixup only)
Model	backbone	ResNet-{18, 34, 50}
	pretrained	True (False in `from_scratch` ablation)
	position head	Conv-BN-ReLU-Conv + soft-argmax on 12×12 grid
	direction head	GAP + FC + Tanh + L2-normalize
	heatmap_size (heatmap variant)	128
	dropout	0.3
Training	optimizer	AdamW
	LR (heads)	1e-3
	LR (backbone)	1e-4 (`backbone_lr_scale = 0.1`)
	weight decay	1e-3
	batch size	8
	max epochs	200
	scheduler	CosineAnnealingLR, `T_max = 200` , `η_min = 1e−7`
	warmup	0
	grad clip norm	1.0
	mixup α	0.4
	position loss	MSE, weight 5
	direction loss	1 − cos sim, weight 5
	early stopping patience	30 on val total loss
Evaluation	bootstrap resamples	1000
	CI level	0.95
	TTA	Off
	wire matching at eval only	K = 2 min-cost assignment

References

Reporting guidelines for AI/ML in medical imaging (recommended by the project assignment):

1. Bluemke DA, Moy L, Bredella MA, et al. *Assessing radiology research on artificial intelligence: a brief guide for authors, reviewers, and readers — from the Radiology Editorial Board.* Radiology 2020; 294(3):487–489. (See also: pubs.rsna.org/doi/10.1148/ryai.240300 — RSNA AI/ML reporting checklist used throughout this report's structure.)
2. Mongan J, Moy L, Kahn CE. *Checklist for Artificial Intelligence in Medical Imaging (CLAIM): A Guide for Authors and Reviewers.* Radiology: Artificial Intelligence 2020; 2(2): e200029.
3. Cohen JF, et al. *AAPM Task Group 273 — Recommendations on the use of artificial intelligence in medical physics.* Medical Physics 2021; 48(8):e857–e872. (10.1002/mp.15170)

Methods cited in the body of this report:

1. He K, Zhang X, Ren S, Sun J. *Deep Residual Learning for Image Recognition.* CVPR 2016. — ResNet-18/34/50 backbones (§3.3).
2. Deng J, Dong W, Socher R, et al. *ImageNet: A large-scale hierarchical image database.* CVPR 2009. — Source of the pretrained weights used for transfer learning (§3.3, §4.3).
3. Zhang H, Cisse M, Dauphin YN, Lopez-Paz D. *Mixup: Beyond Empirical Risk Minimization.* ICLR 2018. — Primary regularizer in the final configuration (§3.6).
4. Kuhn HW. *The Hungarian method for the assignment problem.* Naval Research Logistics Quarterly 1955; 2(1–2):83–97. — Reference for the K = 2 matcher used at evaluation time (§3.5).
5. Loshchilov I, Hutter F. *Decoupled Weight Decay Regularization (AdamW).* ICLR 2019. — Optimizer used throughout (§3.6).
6. Loshchilov I, Hutter F. *SGDR: Stochastic Gradient Descent with Warm Restarts.* ICLR 2017. — Cosine annealing schedule (§3.6).
7. Newell A, Yang K, Deng J. *Stacked Hourglass Networks for Human Pose Estimation.* ECCV 2016. — Conceptual origin of the heatmap + soft-argmax approach used in the heatmap variant (§3.4).
8. Ronneberger O, Fischer P, Brox T. *U-Net: Convolutional Networks for Biomedical Image Segmentation.* MICCAI 2015. — Architectural reference for the skip-connection decoder of the heatmap variant (§3.4).
9. Efron B, Tibshirani RJ. *An Introduction to the Bootstrap.* Chapman & Hall, 1993. — Bootstrap confidence intervals (§4.1).
10. Wilcoxon F. *Individual comparisons by ranking methods.* Biometrics Bulletin 1945; 1(6):80–83. — Paired signed-rank significance test (§4.1, §4.3).

Clinical context cited in §5.4:

1. Routt MLC, Simonian PT, Mills WJ. *Iliosacral screw fixation: early complications of the percutaneous technique.* Journal of Orthopaedic Trauma 1997; 11(8):584–589. — Trajectory tolerance for percutaneous iliosacral screw placement.

2. Mendel T, Noser H, Wohlrab D, Stock K, Brehme K. *The lateral sacral triangle — a decision support for secure transverse sacroiliac screw insertion.* Injury 2011; 42(10):1164–1170. — Updated tolerance reference for sacroiliac screw placement.